MLOPS ASSIGNMENT - 5 REPORT

Github Link -

https://github.com/Layaa-V/MLOps-LayaaVishwakarma-M25CSA017/tree/Assignment-5

Hugging Face Link -

https://huggingface.co/Layaa-V/dlops-assignment5-weights/tree/main

Contents present in this link -

- **best_optuna_model.pt**: The optimal ViT-S model fine-tuned with LoRA on CIFAR-100, discovered during the Optuna hyperparameter search.
- **detector_bim_best.pt**: The ResNet-34 binary classifier trained to detect Basic Iterative Method (BIM) adversarial images.
- **detector_pgd_best.pt**: The ResNet-34 binary classifier trained to detect Projected Gradient Descent (PGD) adversarial images.
- **resnet18_cifar10_best.pt**: The baseline ResNet18 victim model trained from scratch on clean CIFAR-10 images.

WanDB Links -

- Question 1 - https://wandb.ai/0201ai211031-iit/DLOps-Ass5-Q1?nw=nwuser0201ai211031
- Question 1 Optuna - https://wandb.ai/0201ai211031-iit/DLOps-Ass5-Q1-Optuna?nw=nwuser0201ai211031
- Question 2 Part (i) - https://wandb.ai/0201ai211031-iit/DLOps-Ass5-Q2i?nw=nwuser0201ai211031
- Question 2 Part (ii) - https://wandb.ai/0201ai211031-iit/DLOps-Ass5-Q2ii?nw=nwuser0201ai211031

Question 1 : Fine-tuning ViT with LoRA on CIFAR-100 :

For this task, a Vision Transformer Small (ViT-S) model, pre-trained on ImageNet, was fine-tuned on the CIFAR-100 dataset. The objective was to compare a baseline model (where only the classification head was trained for 100 classes) against models utilizing Low-Rank Adaptation (LoRA).

Training Results :

The training results for the final best model are shown below in a tabular form, for all 10 epochs that were run -

Epochs	Training Loss	Validation Loss	Training Accuracy	Validation Accuracy
1	0.5933	0.4052	0.8355	0.8778
2	0.2961	0.3836	0.9064	0.8866
3	0.2198	0.3636	0.9281	0.8928
4	0.1662	0.3913	0.9438	0.8897
5	0.1227	0.3986	0.9592	0.8923
6	0.0869	0.3926	0.9705	0.8968
7	0.0566	0.3976	0.9822	0.9001
8	0.0408	0.3913	0.9880	0.9017
9	0.0321	0.3950	0.9912	0.9025
10	0.9025	0.3925	0.9933	0.9012

Comprehensive Testing Results :

Experiments were performed across various combinations of Rank and Alpha. The baseline model (without LoRA) was also evaluated for comparison. The results are shown below -

LoRA layers (with/without)	Rank	Alpha	Dropout	Overall Test Accuracy	Trainable Parameters used
without	-	-	-	0.8126	38,500
with	2	2	0.1	0.8952	93,796
with	2	4	0.1	0.8942	93,796
with	2	8	0.1	0.8984	93,796
with	4	2	0.1	0.8984	149,092
with	4	4	0.1	0.8997	149,092
with	4	8	0.1	0.8975	149,092
with	8	2	0.1	0.9013	259,684
with	8	4	0.1	0.8994	259,684
with	8	8	0.1	0.9012	259,684

Optuna Hyperparameter Tuning :

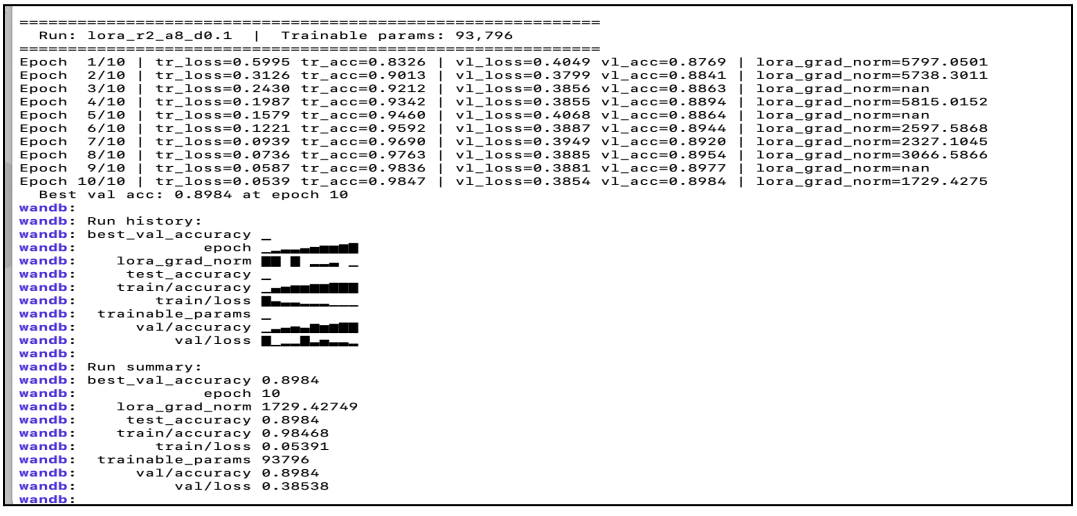
To further refine the LoRA configuration, an Optuna study was conducted to find the optimal hyperparameter combination specifically for the LoRA adapters.

- Best Configuration Found:
    - Rank: 16
    - Alpha: 8
    - Dropout: 0.2
- Corresponding Validation Accuracy : 0.897
- The weights for this optimal Optuna model have been pushed to HuggingFace (link given above).

Analysis & Observations -

- Performance comparison (with and without LoRA) : The baseline model (only training the classification head) achieved a test accuracy of 81.26%. Introducing LoRA adapters immediately boosted the accuracy to roughly 89.5% across almost all configurations, peaking at over 90%.
- Parameter Efficiency : The baseline model used only 38,500 trainable parameters. While the largest LoRA configuration increased this to 259,684 parameters, this represents an incredibly small fraction of the total parameters in a standard ViT-S model (approx. 22 Million), demonstrating LoRA's ability to achieve full fine-tuning performance efficiently.
- Rank - Accuracy relation : Increasing the Rank from 2 to 8 generally provided slight improvements in accuracy (from ~0.895 to ~0.901), indicating that a higher-rank approximation captured more complex task-specific features, though with diminishing returns.

Some Screenshots from my terminal during the experiment -



```
============================================================
  Run: lora_r2_a8_d0.1  |  Trainable params: 93,796
============================================================
Epoch  1/10 | tr_loss=0.5995 tr_acc=0.8326 | vl_loss=0.4049 vl_acc=0.8769 | lora_grad_norm=5797.0501
Epoch  2/10 | tr_loss=0.3126 tr_acc=0.9013 | vl_loss=0.3799 vl_acc=0.8841 | lora_grad_norm=5738.3011
Epoch  3/10 | tr_loss=0.2430 tr_acc=0.9212 | vl_loss=0.3856 vl_acc=0.8863 | lora_grad_norm=nan
Epoch  4/10 | tr_loss=0.1987 tr_acc=0.9342 | vl_loss=0.3855 vl_acc=0.8894 | lora_grad_norm=5815.0152
Epoch  5/10 | tr_loss=0.1579 tr_acc=0.9460 | vl_loss=0.4068 vl_acc=0.8864 | lora_grad_norm=nan
Epoch  6/10 | tr_loss=0.1221 tr_acc=0.9592 | vl_loss=0.3887 vl_acc=0.8944 | lora_grad_norm=2597.5868
Epoch  7/10 | tr_loss=0.0939 tr_acc=0.9690 | vl_loss=0.3949 vl_acc=0.8920 | lora_grad_norm=2327.1045
Epoch  8/10 | tr_loss=0.0736 tr_acc=0.9763 | vl_loss=0.3885 vl_acc=0.8954 | lora_grad_norm=3066.5866
Epoch  9/10 | tr_loss=0.0587 tr_acc=0.9836 | vl_loss=0.3881 vl_acc=0.8977 | lora_grad_norm=nan
Epoch 10/10 | tr_loss=0.0539 tr_acc=0.9847 | vl_loss=0.3854 vl_acc=0.8984 | lora_grad_norm=1729.4275
  Best val acc: 0.8984 at epoch 10
wandb:
wandb: Run history:
wandb: best_val_accuracy _
wandb:             epoch ▁▂▃▄▅▆▇▇▇▇
wandb:    lora_grad_norm ▆▇▁▅▁
wandb:     test_accuracy _
wandb:    train/accuracy ▁▂▃▄▅▆▇▇▇▇
wandb:        train/loss ▇▅▄▃▂▂▁▁▁▁
wandb:  trainable_params _
wandb:      val/accuracy ▁▂▃▄▅▆▇▇▇▇
wandb:          val/loss ▇▅▄▃▂▂▁▁▁▁
wandb:
wandb: Run summary:
wandb: best_val_accuracy 0.8984
wandb:             epoch 10
wandb:    lora_grad_norm 1729.42749
wandb:     test_accuracy 0.8984
wandb:    train/accuracy 0.98468
wandb:        train/loss 0.05391
wandb:  trainable_params 93796
wandb:      val/accuracy 0.8984
wandb:          val/loss 0.38538
wandb:
```

```
============================================================
  Run: lora_r8_a2_d0.1  |  Trainable params: 259,684
============================================================
Epoch  1/10 | tr_loss=0.5950 tr_acc=0.8369 | vl_loss=0.3970 vl_acc=0.8805 | lora_grad_norm=1347.4690
Epoch  2/10 | tr_loss=0.2974 tr_acc=0.9054 | vl_loss=0.3642 vl_acc=0.8880 | lora_grad_norm=1375.0736
Epoch  3/10 | tr_loss=0.2314 tr_acc=0.9254 | vl_loss=0.3771 vl_acc=0.8875 | lora_grad_norm=nan
Epoch  4/10 | tr_loss=0.1786 tr_acc=0.9412 | vl_loss=0.3887 vl_acc=0.8896 | lora_grad_norm=702.4539
Epoch  5/10 | tr_loss=0.1383 tr_acc=0.9535 | vl_loss=0.3822 vl_acc=0.8932 | lora_grad_norm=981.7157
Epoch  6/10 | tr_loss=0.1050 tr_acc=0.9651 | vl_loss=0.3847 vl_acc=0.8972 | lora_grad_norm=1292.0091
Epoch  7/10 | tr_loss=0.0817 tr_acc=0.9733 | vl_loss=0.3857 vl_acc=0.8974 | lora_grad_norm=1155.1784
Epoch  8/10 | tr_loss=0.0651 tr_acc=0.9801 | vl_loss=0.3814 vl_acc=0.9000 | lora_grad_norm=1929.5109
Epoch  9/10 | tr_loss=0.0563 tr_acc=0.9833 | vl_loss=0.3815 vl_acc=0.9024 | lora_grad_norm=nan
Epoch 10/10 | tr_loss=0.0509 tr_acc=0.9858 | vl_loss=0.3807 vl_acc=0.9013 | lora_grad_norm=914.8366
  Best val acc: 0.9024 at epoch 9
wandb:
```

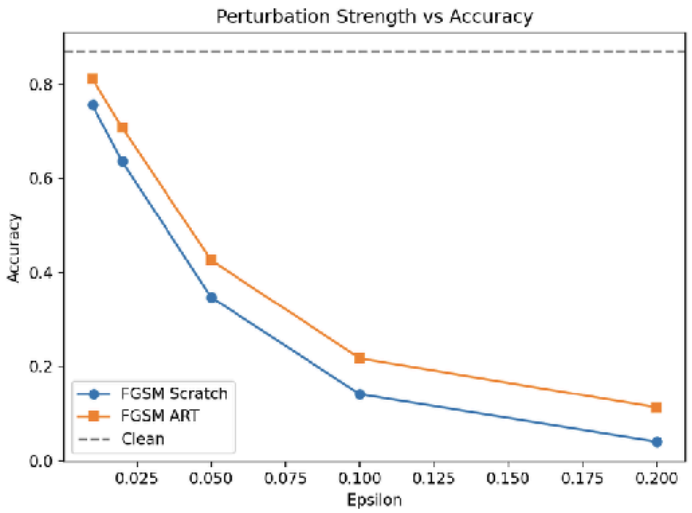
Question 2: Adversarial Attacks using IBM ART

Task (i): FGSM Attack (Scratch vs. IBM ART) -

A standard, non-pretrained ResNet18 model was trained from scratch on clean CIFAR-10 data. Following training, Fast Gradient Sign Method (FGSM) attacks were generated manually (from scratch) and using the IBM Adversarial Robustness Toolbox (ART).

The following graph is a screenshot from the WanDB dashboard, representing the epsilon v/s accuracy readings. The table alongside represents the values clearly.

Epsilon	Accuracy (FGSM Scratch)	Accuracy (FGSM ART)
0.02	0.6364	0.7086
0.05	0.3474	0.4263
0.10	0.1417	0.2179
0.20	0.0405	0.1136



Analysis -

As epsilon increases, the performance of the ResNet18 model drops significantly, demonstrating its vulnerability to adversarial noise.The manually implemented FGSM was slightly more successful at degrading model accuracy than the IBM ART implementation at all epsilon levels. At epsilon = 0.20, the model's accuracy was effectively reduced to random guessing.

Task (ii): Adversarial Detection Model -

Two separate ResNet-34 models were trained as binary classifiers to detect whether an input image was "Clean" or "Adversarial". The adversarial images were generated using the victim ResNet18 model from Task (i). The attacks used were Projected Gradient Descent (PGD) and the Basic Iterative Method (BIM), both implemented via IBM ART.

Detection Performance:

- PGD Detection Accuracy: 85.89%
- BIM Detection Accuracy: 85.19%

Analysis -

Both detectors successfully gained satisfactory accuracy. The detection performance was highly consistent across both the PGD and BIM attacks (85.89% vs 85.19%). This indicates that both iterative attacks generate perturbations that follow similar, detectable patterns in the image space, which the heavier ResNet-34 architecture can learn to distinguish from natural CIFAR-10 images.